SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Transformer Mechanics Worksheet

Course Code:	CSA6502	**Course Title:**	Generative AI and Large-Scale Model
CO Assessed:	CO1-Imitation (BL3)	**Assessment:**	Assessment Tool 3– **Transformer Mechanics Worksheet**
Weightage:	10% of CIA	**Total Marks:**	20
CO1 Statement:	Explain the fundamentals of Artificial Intelligence, Generative AI, Foundation Models, Transformer architecture, tokenization, embeddings, and Large Language Models.		
Student Name:	V Rohan	**Reg. No.:**	192472227
Section / Batch:	CSE-AI	**Date:**	22 July 2026

Code of Conduct

I __V Rohan – 192472227__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___Rohan_____

Question 1 — Tokenization

The word “INTERNATIONALIZATION” is uncommon enough that a sub-word tokenizer would likely split it into smaller pieces rather than keep it whole as a single vocabulary entry. It is, in fact, a favourite example in NLP courses because it is built from several stacked morphemes.

(a) A reasonable 3-piece split

A linguistically motivated split breaks the word along its morpheme boundaries — the smallest meaningful units that combine to build the full word:

Piece	Type	Role in the word
INTER	Prefix	“Between / among” — sets up a relational meaning before the root
NATIONAL	Root / stem	Carries the core meaning, itself already built from “nation” + “-al”
IZATION	Suffix	Turns an adjective into an abstract noun describing a process (“the act of making ___”)

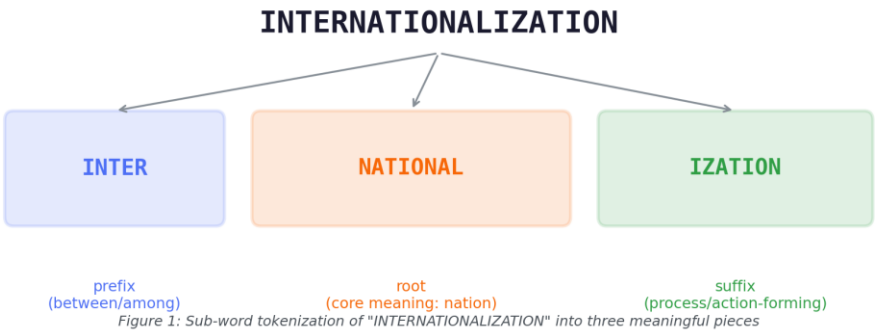

Figure 1: Sub-word tokenization of "INTERNATIONALIZATION" into three meaningful pieces

Figure 1: “INTERNATIONALIZATION” decomposed into prefix + root + suffix, mirroring how a Byte-Pair-Encoding (BPE) or WordPiece tokenizer would greedily merge frequent sub-word units learned from a training corpus.

This is not the only valid split. A statistically driven BPE/WordPiece vocabulary, built purely from character-pair merge frequencies rather than linguistic theory, might instead produce something like “INTERN” + “ATION” + “ALIZATION”, or even four-plus pieces such as “INTER” + “NATION” + “AL” + “IZATION”, depending on the merge rules learned during vocabulary construction. Full marks require only that the pieces (i) concatenate back to the original word, (ii) look like plausible tokenizer output, and (iii) come with a stated justification (frequency, morphology, or merge order).

(b) Why sub-word tokenization helps

Sub-word tokenization lets the model represent an unseen word compositionally — as a combination of familiar smaller pieces it has already learned during training — instead of needing to have memorized the exact whole word “INTERNATIONALIZATION” itself. Because “INTER-”, “NATIONAL”, and “-IZATION” each occur across many other common words (“internet”, “nationality”, “modernization”, “civilization”), the model can transfer the meaning and usage patterns it already learned for those pieces to correctly interpret this long, rare, compound word.

- Avoids the out-of-vocabulary (OOV) problem: whole-word tokenizers must map any unseen word to a single generic <UNK> token, discarding all of its meaning.

- Keeps the vocabulary compact: a fixed set of tens of thousands of sub-word units can reconstruct virtually any long or rare word, instead of storing millions of whole-word entries.
- Generalizes across morphology: learning "-IZATION" once (as "process of making") transfers automatically to "modernization", "globalization", and "internationalization" alike.

Question 2 — Embeddings

Consider the word pairs "king" & "queen", and "king" & "apple".

(a) Which pair has higher cosine similarity?

The pair **"king" and "queen"** would be expected to have the higher cosine similarity.

Embedding vectors are learned so that words appearing in similar contexts end up close together in vector space. "King" and "queen" are both royal titles, both refer to people, and both appear in very similar surrounding contexts ("the ___ ruled the kingdom", "the ___ wore a crown"), so training pushes their vectors to point in almost the same direction, giving a small angle between them and hence a cosine similarity close to 1.

"King" and "apple", on the other hand, belong to entirely unrelated semantic categories — one is a human societal role, the other is a fruit. They essentially never appear in the same kinds of sentences, so their embedding vectors end up pointing in very different directions, giving a much lower cosine similarity. This illustrates that embeddings organize words by shared meaning and usage, not by any surface property such as word length or starting letter.

Figure 2: Cosine similarity in embedding space
(king-queen angle is much smaller than king-apple angle)

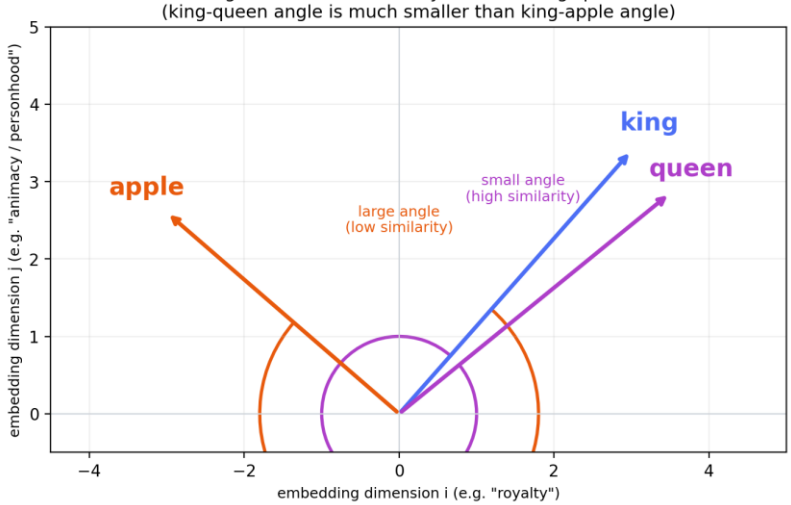

Figure 2: In embedding space, the angle between "king" and "queen" is much smaller than between "king" and "apple", so cosine similarity (which depends only on the angle) is higher for the semantically related pair.

(b) What an embedding represents

An embedding is a dense, real-valued vector (typically a few hundred to a few thousand dimensions) that represents a word's meaning as a position in a continuous, learned vector space. Individual dimensions rarely map to one clean human-readable feature, but the overall geometry of the space captures semantic and syntactic structure: related words cluster together, and consistent directions in the space can encode relationships — the well-known example being that vector("king") − vector("man") + vector("woman") lands close to vector("queen").

This is fundamentally different from a one-hot vector, which represents a word as a single sparse vector the size of the entire vocabulary, containing a 1 at that word's index and 0s everywhere else. Key differences:

- Dimensionality: a one-hot vector has as many dimensions as the vocabulary (often tens of thousands); an embedding is dense and compact (e.g. 300–4096 dimensions).
- Similarity: any two distinct one-hot vectors are equally and maximally dissimilar (orthogonal) by construction — "king" and "queen" would be just as "different" as "king" and "apple". Embeddings instead encode graded, meaningful similarity.
- Learnability: one-hot vectors are fixed, arbitrary identifiers assigned by vocabulary order; embeddings are learned from data so that geometric closeness reflects real semantic or functional closeness.

Question 3 — Self-Attention Worked Example

Sentence: "The city councilmen refused the demonstrators a permit because they feared violence."

(a) What does "they" refer to?

Answer: **"councilmen"** *(circled, as opposed to "demonstrators")*

Because the councilmen are the ones who refused the permit, common-sense reasoning tells us that it is the councilmen who feared violence — that fear is precisely their stated motive for refusing. So "they" should attend most strongly to "councilmen".

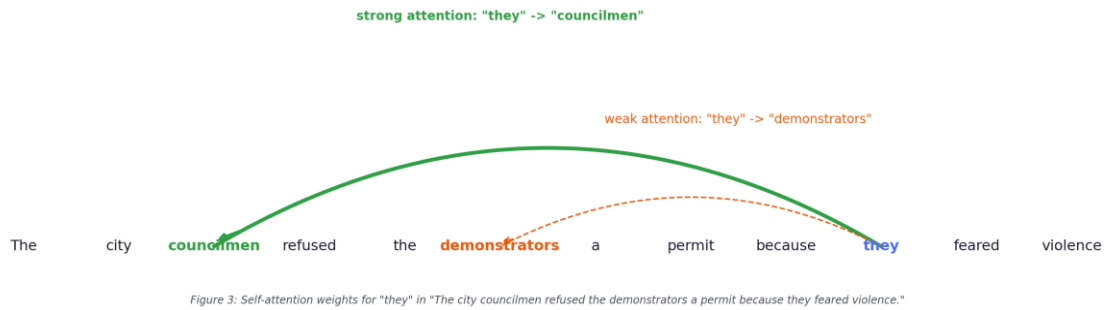

Figure 3: The attention weight from "they" to "councilmen" (solid green arc) should be much stronger than the weight from "they" to "demonstrators" (dashed orange arc).

(b) Why self-attention, not word-order rules, is needed

A simple word-order or proximity rule (such as "pronouns refer to the nearest preceding noun phrase") would incorrectly resolve "they" to "demonstrators", since "demonstrators" sits closer to "they" in the raw sequence than "councilmen" does. Correctly resolving the reference instead requires combining information from across the whole clause — particularly the causal connective "because" and the verb "refused" — together with world knowledge about who typically fears violence in a permit-refusal scenario (the authority granting or denying the permit, not necessarily the demonstrators requesting it).

Self-attention is exactly the mechanism that supports this: every word's representation is updated by looking at, and weighting, every other word in the sentence — regardless of distance — based on learned relevance rather than fixed position. This lets the model dynamically route information from "refused" and "because" back into the representation of "they", correctly linking it to "councilmen" even though "demonstrators" is nearer in the sequence. This sentence, also drawn from the Winograd Schema Challenge, is a classic test case precisely because it defeats simple positional heuristics and demands genuine contextual and pragmatic reasoning — changing just the verb ("feared" vs. "advocated") flips the correct referent entirely, which a fixed rule could never capture.

Question 4 — Query, Key, Value

Self-attention computes, for every token, a new representation built from a weighted combination of all tokens in the sequence. This weighting is computed using three learned linear projections of each token's embedding: the Query, Key, and Value vectors.

Vector	Intuition	Role in the computation
Query (Q)	"What am I looking for?" — represents the current token's question about the rest of the sequence	Compared against every Key to measure relevance
Key (K)	"What do I contain / advertise?" — represents what each token has to offer	Dot-producted with the Query to produce a raw attention score
Value (V)	"What information do I actually pass on?" — the substantive content of each token	Combined (weighted sum) using the normalized attention scores to build the output

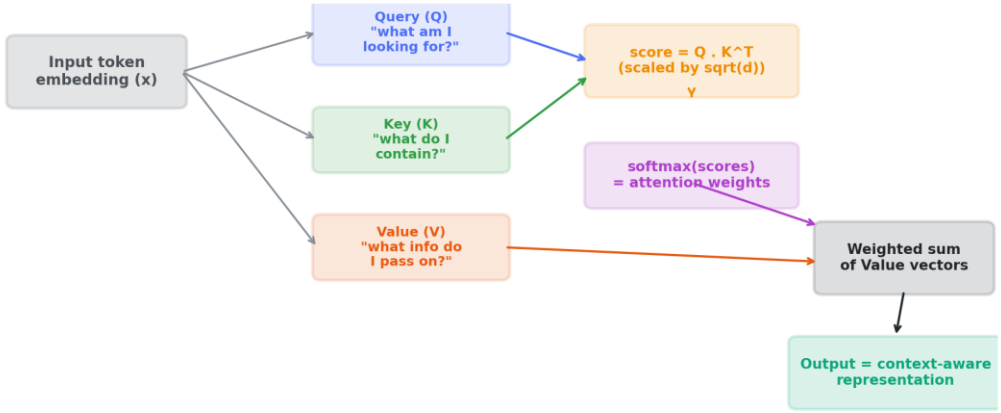

Figure 4: How Query, Key, and Value vectors combine to produce an attention output

Figure 4: The Query of a token is compared against the Keys of every token (dot product), scaled and passed through softmax to get attention weights, which then weight a sum over the Value vectors to produce the output.

How they combine

- Step 1 — Score: for a given token (e.g. "they"), its Query vector is dot-producted with the Key vector of every other token (including itself), producing one raw similarity score per token pair.
- Step 2 — Scale: the scores are divided by the square root of the key dimension ($\sqrt{d_k}$) to keep gradients stable during training.
- Step 3 — Normalize: a softmax is applied across all the scaled scores for that Query, converting them into attention weights that are non-negative and sum to 1.
- Step 4 — Aggregate: the output for that token is the weighted sum of every token's Value vector, using those softmax weights — so tokens whose Key strongly matched the Query (e.g. "councilmen" and "refused") contribute the most Value content to the final result.

In short: Query and Key together decide "how much attention to pay to what", while Value supplies "the actual content that gets passed along" once that attention distribution has been decided. This is what allows the model, in the councilmen/demonstrators example above, to let the representation of "they" be updated mostly by the Value vector of "councilmen" rather than "demonstrators".

Rubrics for Calculation

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical / Concept	30%	Accurately explains tokenization, embeddings, and self-attention mechanics using correct terminology	Explains most mechanics correctly with minor gaps	Partial or superficial understanding of the mechanics	Major misunderstanding of core mechanics
Application	25%	Correctly traces a worked example (e.g. computing attention relevance) step-by-step	Traces most steps correctly with minor errors	Significant errors when applying mechanics to the worked example	Cannot apply the mechanics to a worked example at all
Quality	20%	Clear, logical, mathematically sound reasoning throughout	Mostly sound reasoning with minor errors	Reasoning has notable gaps or inconsistencies	Reasoning is disorganized or largely incorrect
Presentation	15%	Neat, well-labeled diagrams/working, easy to follow	Reasonably clear presentation of working	Presentation is unclear in places	Difficult to follow presentation of working
Documentation / Communication	10%	Shows all work with clear justification for every step	Shows most work with adequate justification	Minimal work shown	No work shown — answers only, unjustified